

RISC-V: ASIC Design and Implementation of a Control Unit for Single-Cycle Microprocessors

Colin Gregg

May 5, 2025

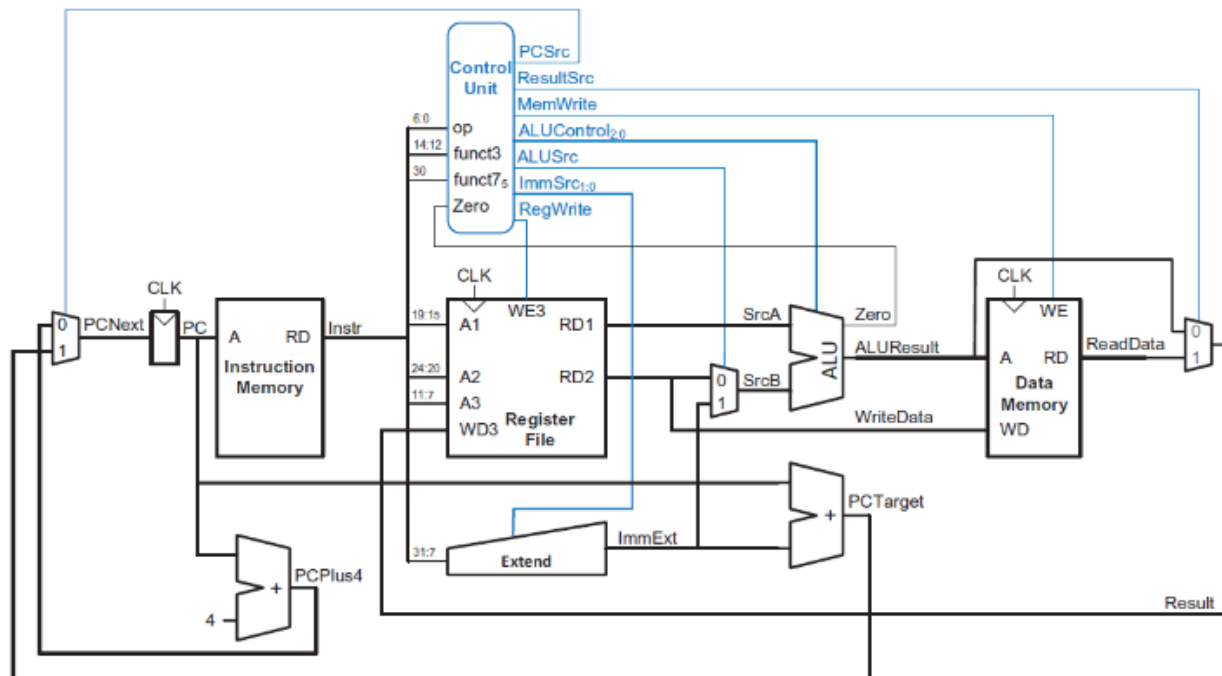
Contents

1	Background	3
1.1	RISC-V	3
1.2	Microprocessors	4
2	Project Description	5
3	Design Specifications	6
4	Design Development	8
4.1	Overview	8
4.2	Creating the ROM and Program Counter Modules	9
4.3	Creating the Control Unit Module: Inputs	10
4.4	Understanding the RISC-V ISA	10
4.4.1	A Digression on Atomic Instructions	13
4.5	Creating the Control Unit: Outputs	14
5	Initial Design	20
5.1	Design Implementation	20
5.1.1	Control Unit Module	21
5.1.2	ROM Read Module	22
5.1.3	Top Module	23
5.2	Test Case Development	23
5.2.1	Creating an Instruction	23
5.2.2	Test Bench	27
5.2.3	Results and Discussion	28
6	Second Design	32
6.1	Changes	32
6.2	Validation	33
6.2.1	Coverage	33
6.3	Synthesis	37
6.3.1	Design	37
6.3.2	Log and Waveform	40
7	Conclusion	46

1 Background

RISC-V Base Microprocessor

Basic microarchitecture for a RISC-V microprocessor [1].



1.1 RISC-V

RISC-V is a highly modular open-standard instruction set architecture (ISA) [2]. It is based on the reduced instruction set computer (RISC) principles which emphasize simplicity and

efficiency through its instructions. The RISC-V ISA is comprised of a core set of instructions, the RV32I instruction set, which perform. Although simple, the ISA can be modified to add extensions, like the atomic (RV32A) instruction set, and, just as easily, can be modified to omit instructions based on what is required for the given problem [2]. This modularity makes it ideal for applications like Systems on Chip (SoC), embedded systems, or high performance computing systems where memory is small and speed is imperative. Additionally, due to its open-nature design and widespread collaboration, RISC-V is ideal for supporting academic projects, and its versatility and specificity in design make it a great learning tool.

1.2 Microprocessors

Every home computer or laptop contains a Central Processing Unit (CPU) which executes instructions given to it by a computer program. However, what most people think of as the CPU is typically a microprocessor connected to the computer's motherboard. A microprocessor is capable of handling both the data processing logic and the control logic, like a CPU, but it is contained to a single integrated circuit chip. By being on a singular chip, the cost for manufacturing is greatly reduced, and reliability is increased since there are fewer interconnects.

A microarchitecture represents the means by which a processor is constructed in order to implement a given ISA. The basic structure of a microprocessor includes a program counter (byte addressed), register file, Arithmetic Logic Unit (ALU), instruction memory (ROM), data memory, and a control unit. Using these components, it is able to completely process an instruction which includes fetching, decoding, and executing the instruction. A RISC-V microprocessor is designed to implement a specific ISA chosen by the designer. The microarchitecture needed to accomplish this may require either more or less hardware.

The processor depicted in Figure 1.1 is a single-clock-cycle implementation of a RISC-V microarchitecture; it is designed to support many of the standard RV32I instructions like branch-if-equal (beq), store word (sw), and a subset of the I-type and R-type instructions. A core component of this architecture is the Control Unit, highlighted in blue in Figure 1.1, which performs the critical step of decoding a fetched instruction and generating the corresponding control signals. These control signals dictate the operation that will be performed by each subsequent piece of hardware in the microarchitecture. The set of signals required for each piece of hardware for a specific instruction are called a data path.

2 Project Description

The goal of this project is to design and implement a control unit that accurately generates control signals for a subset of the RV32I instructions, along with the complete set of RV32A (atomic) instructions. The generic single-cycle micro-architecture introduced earlier does not completely support these specific requirements. To develop a control unit that meets the given objectives, there are several key issues inherent in the base design that must be addressed:

- **Limited Hardware Support for Atomic Operations.** The default architecture does not include the additional hardware necessary for implementing atomic instructions. These instructions will need a checking and lock system to operate properly.
- **Lack of Encoding for Control Signals.** Because the instruction set for this project is highly specific, the standard encoding provided for the basic architecture will not satisfy our needs. A more robust and larger encoding scheme will be necessary for the control unit being designed.
- **Overly Broad Baseline Design.** The generic architecture, as expected by a generic design, includes components and functionalities that are not necessary for implementing our specific subset of instructions.

Successfully fulfilling the given Project Requirements Document (PRD) requires designing the control unit to take advantage of the intentionally lean and modular design of RISC-V. Only the necessary control signals will be taken into account when creating an encoding scheme and designing the hardware needed. I will assume all other hardware in the microarchitecture proposed above remains the same, which will be integral in creating data paths for different instructions, as seen later. However, I will need to account for the fact additional hardware (a register) will be needed to implement the RV32A instructions.

3 Design Specifications

The purpose of a Control Unit is to characterize the topology of the data paths which will implement the desired instruction set for a micro-architecture. Therefore, we must define what our system will implement in order to clarify how the control unit must be designed. The design for this project is based on the Product Requirements Document (PRD) given to us. The PRD specifies the following:

PRD Requirement

Design the appropriate control unit which will read instructions from a memory block and process it through the principles of the given ISA to create the appropriate control signals.

The memory block implemented will be a 17-word long, hard-coded, block of ROM which stores 32 bit instructions. The ISA chosen will be the following subset of the RISC-V ISA:

All RV32A Atomic Instructions,

- lr.w
- sc.w
- amoswap.w
- amoadd.w
- amoand.w
- amoor.w
- amoxor.w
- amomax.w
- amomin.w

and the following RV32I Instructions:

- addi
- ori
- andi
- lw
- sw

The designed control unit will read instructions from the ROM-resident code, process the fetched instruction based on the given RISC-V ISA, and generate the appropriate control signals and register selections.

4 Design Development

4.1 Overview

Development of the Control Unit begins by breaking down the three main tasks it will do:

- Read instructions from ROM
- Process each instruction
- Output control signals based on the instruction

Three modules are necessary to accomplish these tasks: a block of Read Only Memory (ROM), a Program Counter to access the ROM, and the control unit. We can represent these objects, and their connections, through the following block diagram:

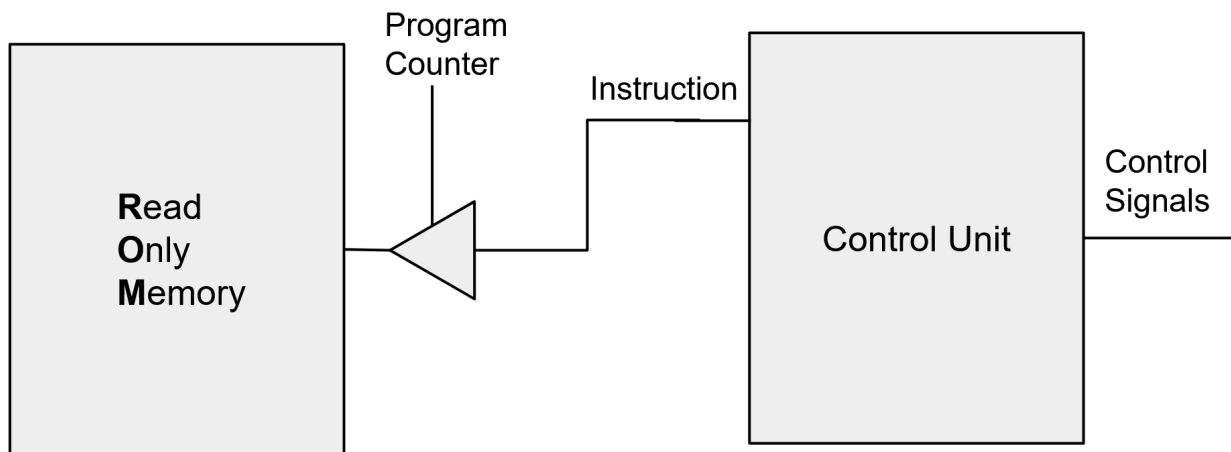


Figure 4.1: High-level top module block diagram.

4.2 Creating the ROM and Program Counter Modules

From diagram 4.1, it is clear that additional hardware for the Program Counter (PC) and the ROM block must be implemented. The ROM block can be implemented in a variety of ways, and it will be seen later that the method chosen changed between the initial and final design. The Program Counter, however, is a very standard implementation. The function of the program counter will be to track which instruction needs to be read, fetch that instruction, and deliver it to the control unit. Because this is a single-cycle processor, the Program Counter will update with every clock cycle, in order to provide a new instruction. Although no Jump instructions are implemented, so the instruction counter will only increment linearly, it will be useful to have a reset capability for the Program Counter. The process of reading from ROM and incrementing the program counter each clock cycle is represented by the following diagram:

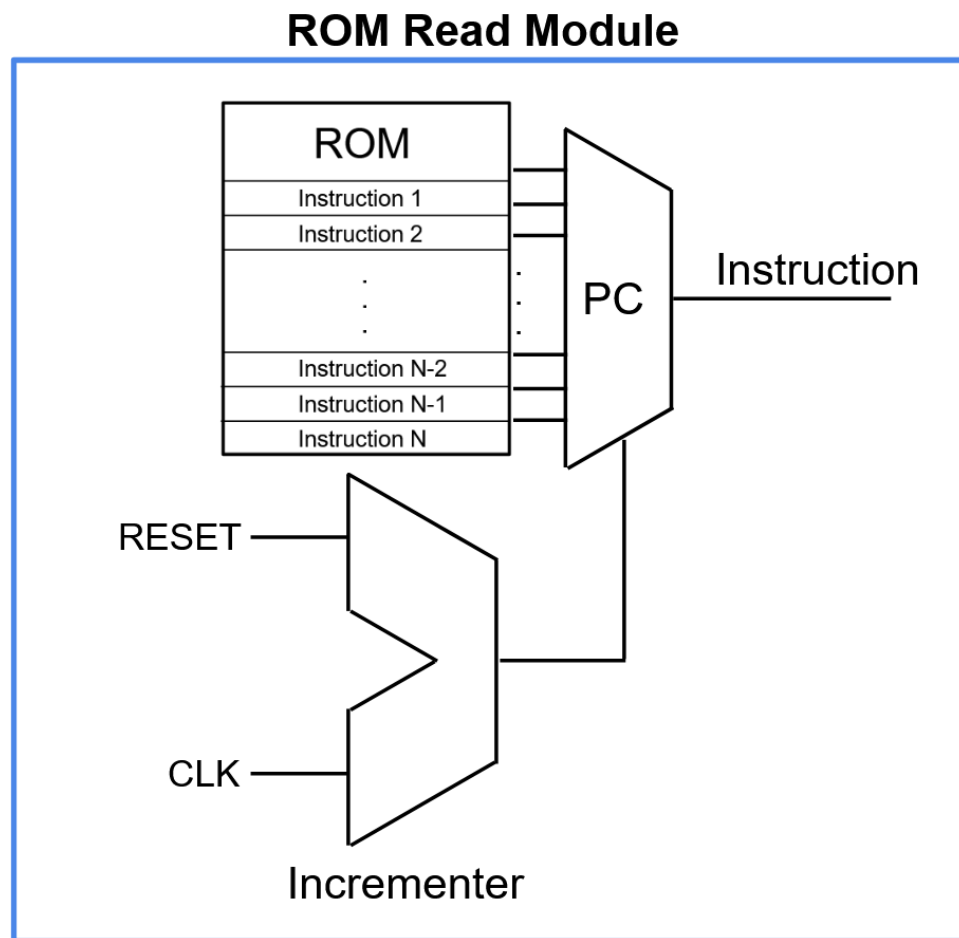


Figure 4.2: Block diagram describing hardware components to read an instruction from ROM.

4.3 Creating the Control Unit Module: Inputs

To design the Control Unit, I will adopt a similar method to that described in the textbook ‘Digital Design and Computer Architecture, RISC-V Edition’ [1]. In that book, the Control Unit is broken down into two decoders. Heuristically, this methodology makes sense: the control unit receives a finite set of instruction types and then select an output based on that input. This function is exactly what a decoder does. Although my design will follow the same architecture, it will have different inputs and outputs. Despite that, the the control unit can be represented, generally, with the following diagram from the textbook:

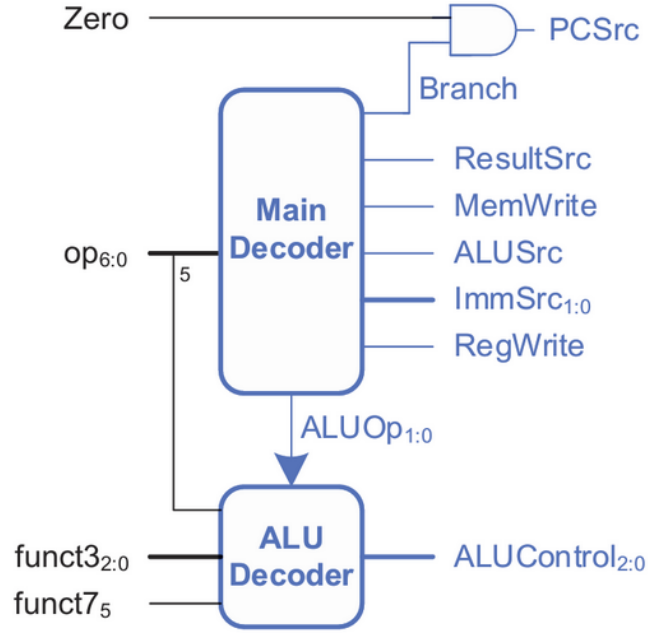


Figure 4.3: Dual Encoder topology of Control Unit [1].

4.4 Understanding the RISC-V ISA

In order to understand how the decoders will work, it is imperative to first define what our instructions are, how they look, and how they are processed.

An Instruction Set Architecture (ISA), broadly speaking, defines how software can control a processor (or family of processors). The software exerts this control by calling instructions. There are two types of ISAs: CISC and RISC. RISC stands for Reduced Instruction Set Computer, and works by only implementing frequently used, simple instructions. The RISC-V ISA, as implied by its name, falls under the RISC classification [3].

An ISA will consist of various types of instructions, like R, I, and S, which describe what action those types of instructions will perform. R-type instruction are ones that only work with data values located in registers. Meanwhile, I-type instructions implement an immediate

value (one taken from the instruction) and a register value [4]. S-type instructions indicate working directly with memory, typically writing an immediate value to local Random Access Memory (RAM) [5].

The RISC-V ISA consists of two categories of instructions, with each category containing instructions of various types. The two categories are:

Base Integer (RV32I) instructions

RV32I instructions form the basis of the RISC-V ISA, and as such, are very extensive. A list of all the instructions is shown below, with the six instructions we will implement highlighted in red, and their format highlighted in blue:

RV32I Base Integer Instructions							
Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srl	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1101111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger	

Figure 4.4: Table of RV32I instructions for the standard RISC-V ISA. Highlighted in red are the required instructions required by the PRD, while the blue highlights the format of each instruction.

Standard Extensions

Standard extensions are sets of instructions that are supported by RISC-V and can be implemented based on the requirements of a project [6]. There are many possible Standard Extensions, but to satisfy the PRD, this control unit must implement the RV32A Atomic

Extension. These instructions can be seen in the figure below. It is noted that they are all "modified" R-type instructions (of the R format, but a different opcode):

RV32A Atomic Extension

Inst	Name	FMT	Opcode	funct3	funct5	Description (C)
lr.w	Load Reserved	R	0101111	0x2	0x02	rd = M[rs1], reserve M[rs1]
sc.w	Store Conditional	R	0101111	0x2	0x03	if (reserved) { M[rs1] = rs2; rd = 0 } else { rd = 1 }
amoswap.w	Atomic Swap	R	0101111	0x2	0x01	rd = M[rs1]; swap(rd, rs2); M[rs1] = rd
amoadd.w	Atomic ADD	R	0101111	0x2	0x00	rd = M[rs1] + rs2; M[rs1] = rd
amoand.w	Atomic AND	R	0101111	0x2	0x0C	rd = M[rs1] & rs2; M[rs1] = rd
amoor.w	Atomic OR	R	0101111	0x2	0x0A	rd = M[rs1] rs2; M[rs1] = rd
amoxor.w	Atomic XOR	R	0101111	0x2	0x04	rd = M[rs1] ^ rs2; M[rs1] = rd
amomax.w	Atomic MAX	R	0101111	0x2	0x14	rd = max(M[rs1], rs2); M[rs1] = rd
amomin.w	Atomic MIN	R	0101111	0x2	0x10	rd = min(M[rs1], rs2); M[rs1] = rd

Figure 4.5: Atomic instructions, with format, opcode, funct3, and funct5 provided.

Instructions come in standard formats based on the type of the instruction. Examining the table below we can see the composition of each instruction type: **RV32I instructions**

	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R-type	funct7				rs2		rs1		funct3		rd		opcode	
I-type	imm[11:0]						rs1		funct3		rd		opcode	
S-type	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	

Figure 4.6: Format for types of RV32I instructions.

RV32A instructions

31	27	26	25	24	20	19	15	14	12	11	7	6	0
funct5			aq	rl	rs2		rs1		funct3		rd		opcode

Figure 4.7: Format for RV32A instructions.

The most important part of each instruction is the opcode: the opcode is always the last seven bits of an instruction, and it conveys what type the instruction is. As mentioned above, our control unit must handle R, I, and S types, however there are two exceptions that must be handled. First is the Load Instruction, which is a special I-type instruction. Like

the Store Instruction, Load has to deal with memory, and therefore the registers specified by its instruction must be operated on differently than a typically I-type instruction. Load and Store both are similar to I-type instructions because they both utilize an immediate value provided by the instruction, but instead of using the immediate value to perform an operation, like addition, load and store use it to specify what position of a register to access. The second exception is the RV32A Instructions. Although deemed an R-type format by the table above, the opcode for these instructions is different from the RV32I R-type instructions. This is because atomic instructions, although dealing with only registers, have an entirely separate functionality which involves locking and checking specific registers.

Instructions are further classified by their functionality, given by the funct3 and funct5 values, which can be found by their bit-location specified in the instruction above. While the opcode defines the type of instruction, the funct values determine which operation the instruction performs. For RV32I I- and S-type instructions, funct3 is what determines the operation. Another necessary piece of information for I-type instructions is the Immediate block: this provides the value that will be used to perform the operation with the stored register value. For RV32A instructions, funct3 is the same for all of them, however, funct5 determines which operation is performed. Lastly, it is important to note that rs2, rs1, and rd refer to the registers that will be accessed to perform each of the operations. RISC-V has 32 given registers to choose from.

4.4.1 A Digression on Atomic Instructions

Atomic instructions deal with the problem of multiple processors accessing the same memory address [7]. They avert this by flagging a specified memory address until all operations have been performed. Atomic instructions can do this in three ways, determined by the input bits aq (acquire) and rl (release). All three methods ensure that the developer can guarantee a value at a register has been fully operated on by the desired memory-accessing instructions before it becomes visible to other hardware threads (harts). Since all RV32A instructions are memory-accessing, and most of them perform operations like add, and, or swap, which can take multiple clock cycles when strung together sequentially, it is necessary to guarantee the address being operated on is not accessed by any other hardware threads in the system.

The first method, when only the aq bit is high, flags the reserved register so any proceeding memory accesses to the reserved register, before the desired memory operations finish, will fail. For example, Load Word Reserved, lr.w, is called on register rs2, then amoadd.w is called. If another hart in the system performs a memory operation on rs2 before amoadd.w finishes and is stored, via sc.w, then that register will be flagged and the operation will fail. The second method, when only rl is high, does essentially the opposite. Again, the reserved address is flagged, and it will fail if there is any memory access to that register before the preceding memory operations finish. If both aq and rl are high, the sequence of memory operations on the reserved address are sequentially consistent and cannot be observed before any earlier memory operations or subsequent memory operations in the same hart finish [8].

4.5 Creating the Control Unit: Outputs

Observing figure 1.1 again, the desired control unit will consist of two decoders and have opcode, funct3, funct5, aq, and rl as their inputs. We can also see from the diagram provided in the PRD that the outputs will include ResultSrc, MemWrite, ALUControl, ALUSrc, ImmSrc, and RegWrite. Based on the design from the textbook, an ALUOp control signal will be needed for the ALU decoder. From the discussion on RV32A instructions, we will need an output for the reserved address and the flag which tracks if the address has been touched. To determine which output combinations correspond to a given input we must observe what external hardware the outputs are connected to, and what behavior they will cause. To do this, I began by observing the diagram for a single-cycle RISC-V microprocessor, as well as a truth table for the main decoder, provided by the book. These can both be seen below.

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000	1	0	1	0	0	0	00

Figure 4.8: Truth table for Main Decoder from textbook [1].

Using the microprocessor diagram and the decoder truth table, I created datapaths for the I-type, load, and store instructions. I did not provide one for the RV32A instructions because the results for those instructions can be easily extrapolated from the current known information. I would also need to add an additional register and flag to the diagram. The data paths are shown below in figures 3.9, 3.10, and 3.11.

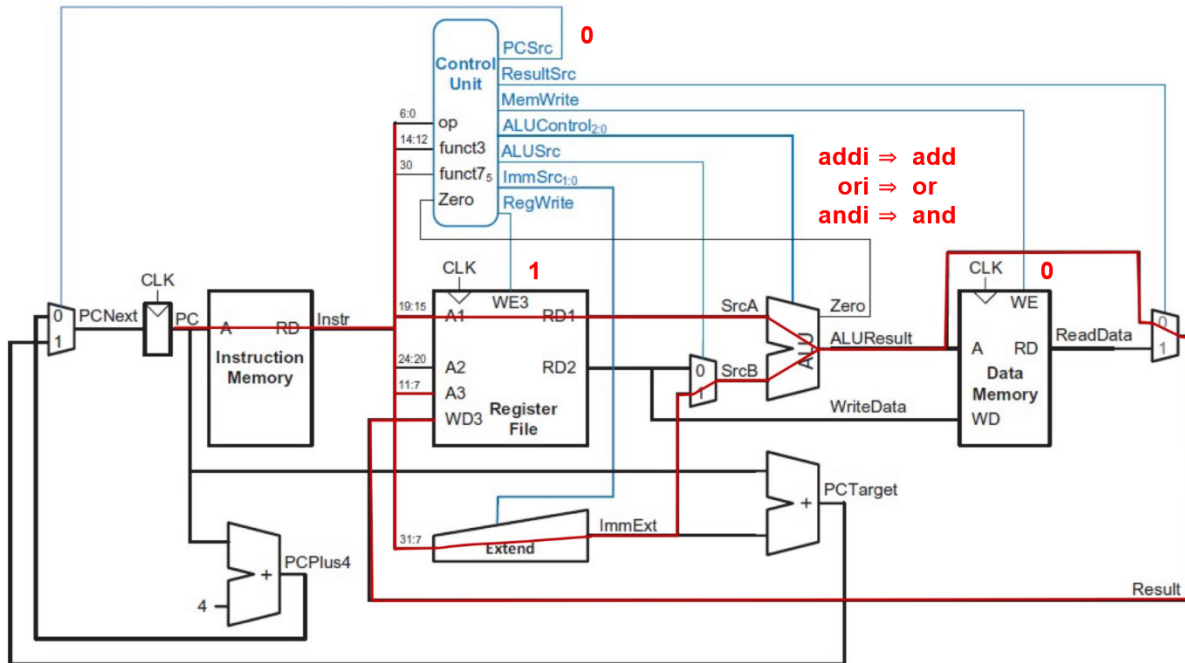


Figure 4.9: I-type Datapath for basic RISC-V microprocessor microarchitecture.

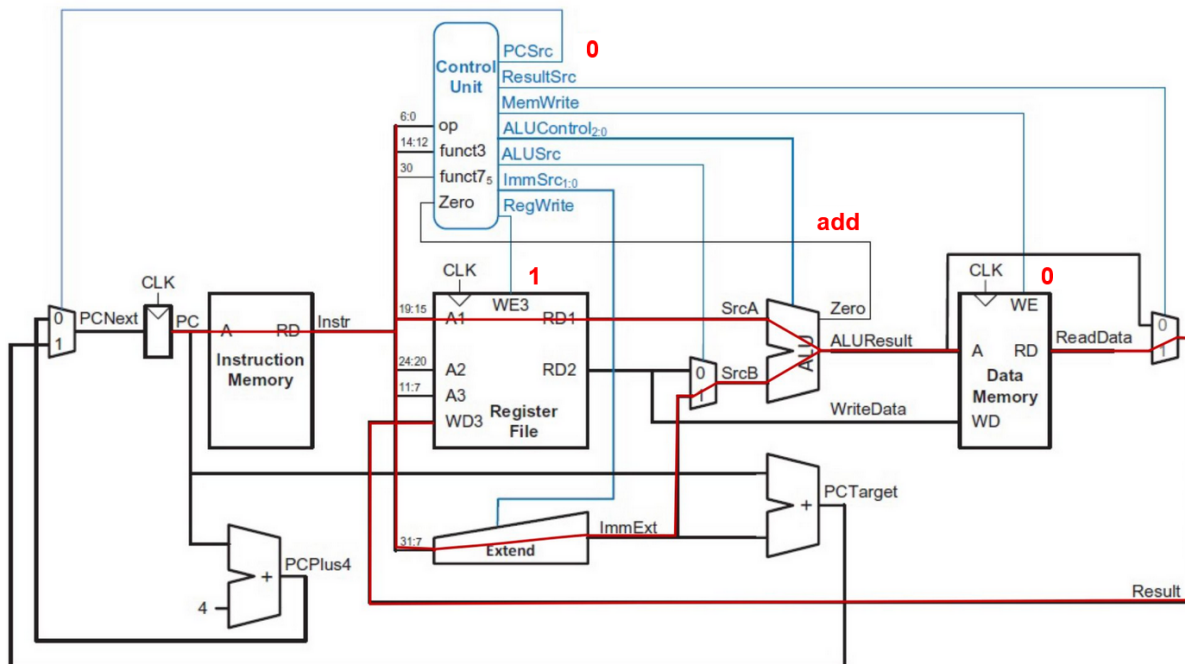


Figure 4.10: Load Word Datapath for basic RISC-V microprocessor microarchitecture.

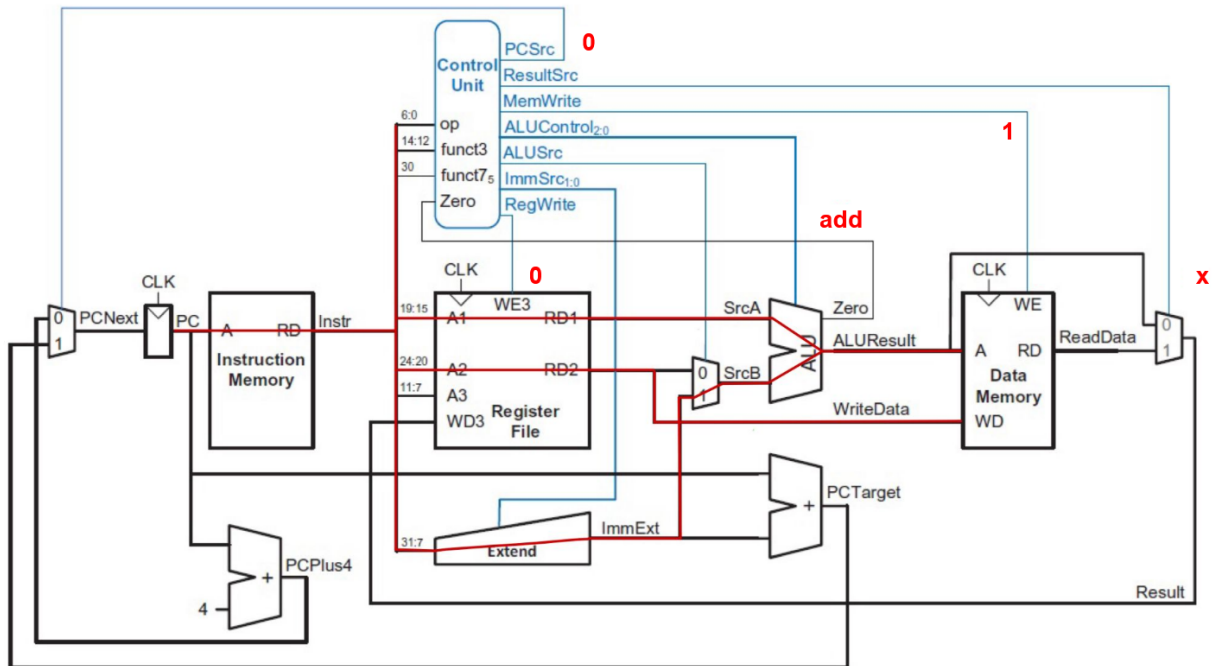


Figure 4.11: Store Word Datapath for basic RISC-V microprocessor microarchitecture.

Combining everything covered so far, I designed the following encoding scheme which maps ALUControl signals to a corresponding operation:

ALU Control Signals

ALUControl	Operation
3'b000	lw, sw
3'b000	add
3'b001	sub
3'b010	and
3'b011	or
3'b100	DO NOT CARE
3'b111	xor

Figure 4.12: Store Word Datapath for basic RISC-V microprocessor microarchitecture.

Note, I only include add, sub, etc..., and not addi or amoad.w, for example. This is

because both the immediate and atomic instructions perform the same operation; they only vary in what registers they access and how. The truth table for the control unit will be broken down into two tables: one for each decoder. The main decoder will take the opcode as its input, and return the outputs which are general for each type of instruction. It will also output an ALUOp which, along with funct3 and funct5, will control the ALU decoder. The ALU decoder will then produce the correct ALUControl signal. This procedure mirrors our prior knowledge that the opcode determines the type of instruction, while funct3 and funct5 determine the operation done. An additional note is that imm, aq, and rl are not represented in the truth tables because they do not affect the decoding of which operation must be done. Instead, they are used later to assist in performing that operation. The truth tables for the main decoder and the ALU decoder can be seen below.

MAIN DECODER

Instruction	Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
Atomic Instr	7'b0101111	1	2'b10	0	0	0	0	2'b10
I-type	7'b0010011	1	2'b00	1	0	0	0	2'b10
lw	7'b0000011	1	2'b00	1	0	1	0	2'b00
sw	7'b0100011	0	2'b01	1	0	0	0	2'b00

Figure 4.13: Truth table for my Main decoder.

ALU Decoder				
ALUOp	funct3	funct5	ALUControl	Instruction
2'b00	0x0		3'b000 (add)	lw, sw
2'b10	0x6		3'b000	addi
2'b10	0x7		3'b011	ori
2'b10	0x2		3'b010	andi
2'b00	0x2	0x02	3'b000	lr.w
2'b00	0x2	0x03	3'b000	sc.w
2'b00	0x2	0x01	3'b000 (add)	amoswap.w
2'b10	0x2	0x00	3'b000	amoadd.w
2'b10	0x2	0x0C	3'b010	amoand.w
2'b10	0x2	0x0A	3'b011 (or)	amoor.w
2'b10	0x2	0x04	3'b111 (xor)	amoxor.w
2'b10	0x2	0x14	3'b001 (sub)	amomax.w
2'b10	0x2	0x10	3'b001 (sub)	amomin.w

Figure 4.14: Truth table for my ALU decoder.

Columns with light blue headers are inputs, columns with a dark blue header are outputs, and columns with purple headers specify the corresponding instruction. Instructions highlighted in orange are RV32I instructions, while those highlighted in yellow are RV32A instructions. The ImmSrc output for all atomic instructions is red because it is a do not care value. However, to aid in synthesis, I removed all do not care (DNC) values, and replaced them with a set of default values. These values will cause the hardware given in the diagrams above to do nothing. The default values I chose are given in the following table.

Default Values		Reason
RegWrite	1'b0	Do not write to register file
ImmSrc	2'b10	Not doing immediate operation so do not need it
ALUSrc	1'b1	Not using immediate value from instruction
MemWrite	1'b0	Not writing to memory
ResultSrc	1'b0	Not Loading from Memory
Branch	1'b0	Do not trigger branch
ALUOp	2'b01	Not used in encoding
ALUControl	3'b100	We do not do anything with this encoding

Figure 4.15: Default value encoding for Control Unit.

Everything covered so far can be combined to create the following block diagram for the control unit module:

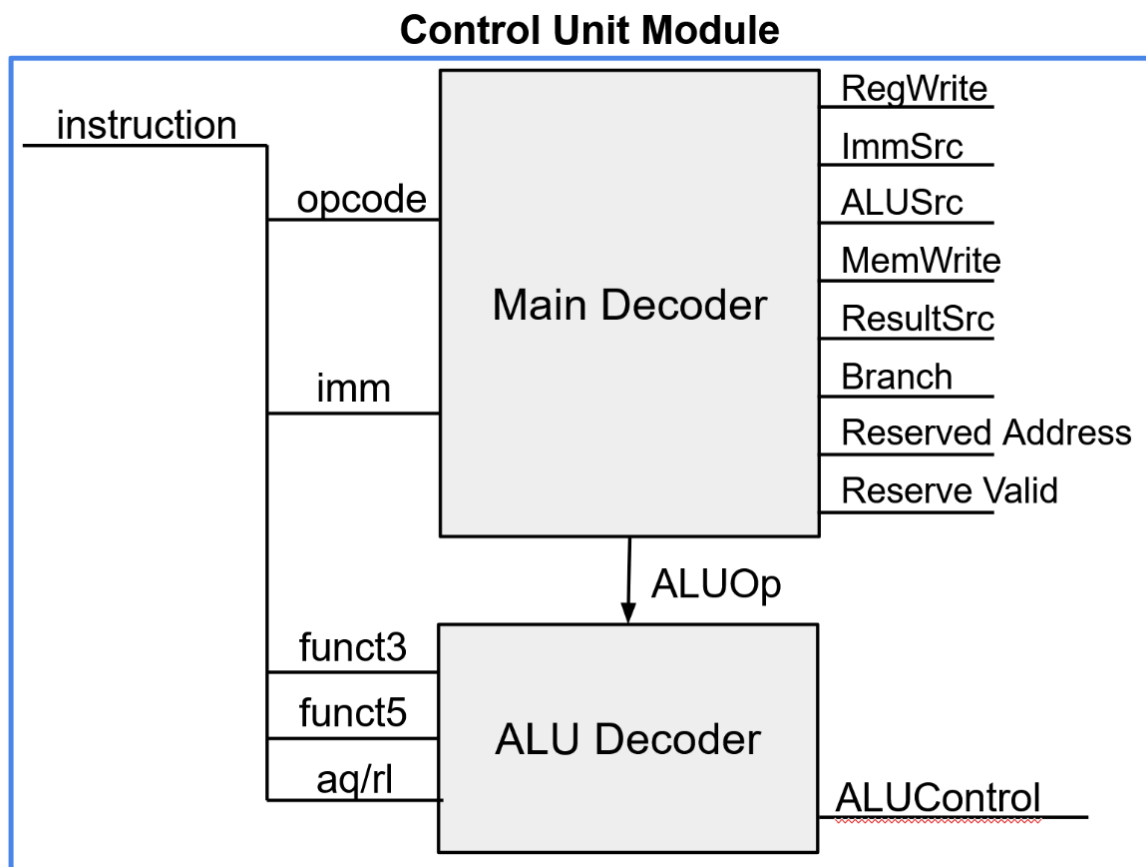


Figure 4.16: Block diagram for Control Unit.

5 Initial Design

5.1 Design Implementation

The initial implementation for the control unit involved creating a starter design and test-bench file, in verilog format, for a rough draft of the logic of our control unit. The goal with this design was to create the high-level logic necessary to implement the two decoders of the control unit, as well as to create a read ROM module. In total, I created three modules that connect in the following format:

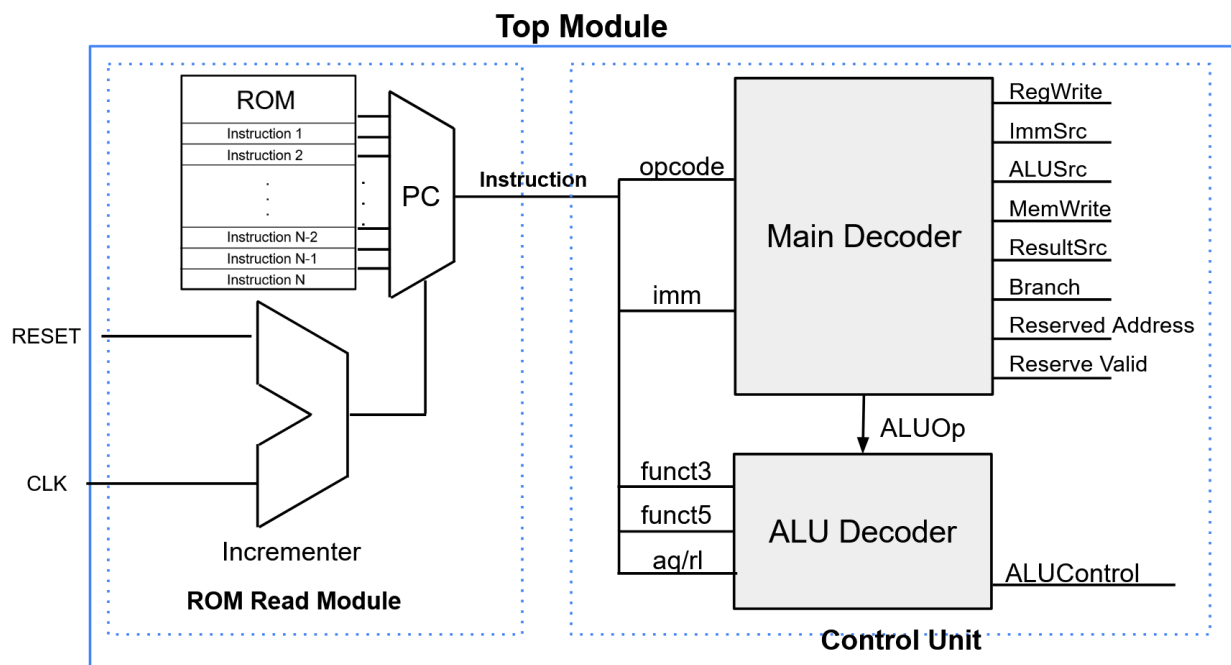


Figure 5.1: Block diagram for entire design file. Shows ROM read and control unit modules instantiate inside a top module environment.

5.1.1 Control Unit Module

A decoder lends itself nicely to being programmed in verilog since its truth table can easily be implemented using a case statement. Two decoders in series then can be programmed using a nested case statement, with the outer one testing the opcode, and the inner one checking the funct3 and funct5 values. As seen from diagram 5.1, a set of outputs will be determined by the type of instruction—outputs RegWrite through Branch—which is handled by the initial case statement. Further outputs requiring additional logic will depend on the operation performed by the instruction. The next several sections discuss in more detail the necessary logic to implement.

Atomic Instructions

Determined by the opcode 7'b0101111, the RV32A instruction set requires additional logic to ensure the checking and locking of the chosen reserve address depending on which instruction is called. The reserve address and flag must be set if the load reserve instruction is called, and they must be checked if the store conditional or swap instructions are called. The rest of the instructions do not require any additional conditions to be set. An if-else statement that checks the funct5 value—which determines the operation—was used to accomplish this functionality. If the funct5 value matches the lr.w, it sets the appropriate ALUOp, ALUControl, reserved address, and reserve flag values. Otherwise, if the funct5 matches with the sc.w instruction it checks the reserve flag: if the flag is unaltered, the store action is performed and the address is un-reserved, otherwise it returns the default ALUControl signal which signals to do nothing. The amoswap.w instruction is also included in the else-if part of the statement because it has the same ALUOp signal as the sc.w, but it does not have any of the additional logic; it only sets the ALUControl signal. A case statement based on funct5 is used to determine which instruction to choose once inside the else-if statement. All other operations for RV32A have an ALUOp signal of 2'b10, and are gathered in the final else part of the if-else if- else statement. Another case statement based on funct5 is used to determine which operation is being performed, and the ALUControl signal is set accordingly.

Immediate Instructions

The next opcode to check is for the I-type instructions, given by 7'b0010011. Again, all outputs except the ALUControl are determined immediately by the main decoder truth table. The only output left to determine is the ALUControl since only the RV32A instructions interact with the reserving functionality. A case statement based on funct3 is used to determine the ALUControl value based on the operation.

Load Instructions

The third case is the Load instruction, given by the opcode 7'b0000011. No additional logic is needed since it is one instruction; the outputs are determined immediately by the main and ALU decoders.

Store Instructions

The last case to check is the store instruction, with an opcode of 7'b0100011. An if statement to check whether the current rs1 register being accessed through the instruction is flagged as the reserve address register allows the reserve flag to be altered if the reserve address is actually accessed. Otherwise every output is set on the main and ALU decoder.

5.1.2 ROM Read Module

The first design had implemented the reading from ROM and program counter functionality into the test bench, which worked for producing the correct log and waveform outputs, however, this produces problems when trying to synthesize the results. The testbench is not synthesizable, so once the first implementation had confirmed the control unit logic was functioning as intended, the ROM read module had to be added to the design file.

The initial ROM read module was created using the ‘readmemh’ function. This reads 32 bit (1 word) lines from a .dat type file, which included a list of instructions. This worked for testing cases, but the Synopsys Design Compiler cannot initialize memory through this method while synthesizing, so another implementation was necessary.

The second version of the ROM read module needed to both read from a set list of instructions that acted as ROM, and also be synthesizable. To satisfy both conditions it was necessary to switch from using the .dat file to hold the instructions list, to hard-coding the instructions into a ROM block in the Verilog design file. This method works, but it has the draw back of needed to hardcode a pre-set list of instructions into the Verilog code, making the control unit not modular at all. The ROM block is synthesized completely as combinational logic, so I cannot image a method to overwrite the ROM, making this control unit rather useless unless it is resynthesized every time with a new ROM block.

32-bit long instructions are used, which becomes very easy to access via a program counter if the ‘incrementer’ hardware (of diagram 4.2) is set to four. This ”byte addressed” the ROM block since one instruction is four bytes long and therefore each instruction’s location can be given by an integer multiple of four. To access each instruction, a program counter value will need to be determined that will exactly match the location in the ROM block. For ease, the ROM block will be assumed to be 32 instructions long; in reality, the ROM block will be implemented as a case statement and therefore only be 17 instructions long, but it is capable of holding 32 instructions. As demonstrated below, the ROM block is byte addressed, each instruction’s location in the block will be an integer multiple of 4. It turns out that the byte address can easily be represented as decimal value that increments by one by bit shifting the byte address to the right by two. Observe the following pattern of decimal numbers represented in binary:

```

0 → 00000
4 → 00100
8 → 01000
12 → 01100
16 → 10000

```

After performing a right bit shift twice, the binary numbers become the following:

```

000 → 0
001 → 1
010 → 2
011 → 3
100 → 4

```

Which when converted back to decimal provides a counting sequence in decimal. The reason to do this is because in typical processors the program counter is always incremented by four, but to reduce the number of digits needed to represent the ROM block since it is being hard coded as combinational logic and each bit needed for the program counter will be an additional wire trace running all across the chip layout. That will lead to lots of unnecessary capacitance, heat dissipation, and delays. Additionally, because the ROM block is set to be 32 instructions long, only five bits are needed to access it completely, so it is known that the program counter can be set to be seven bits long. Since only the five most significant bits are being used, the Design Compiler will know to route those to the ROM block and ignore the first two (which could be used for another ROM block or possibly RAM), and a bit-shifter is not actually needed, saving space, time, and power. Lastly, it has been mentioned in this section, but Design Compiler synthesizes always blocks as combinational logic, so representing the ROM block as an always block with a case statement that uses blockin assignments for each instruction will be easily synthesized by the compiler.

5.1.3 Top Module

Because both the Control Unit and Read ROM modules must be connected and run with every clock cycle, a top module is needed to instantiate and wire these modules. The top module takes the clock and reset ports as its inputs, and its outputs are the combination of the outputs for the Control Unit and Read ROM modules.

5.2 Test Case Development

It would be needless to create instructions that run an actual program since the control unit is not attached to any hardware, as such it is only important to check that the correct control signals are being output from the module. For this reason, the test instructions will consist of the set containing all required instructions by the PRD. There will be one exception, where a series of additional store, lw.r, and sc.w instructions are used to verify the functionality of the RV32A instructions.

5.2.1 Creating an Instruction

When designing an instruction, it can be useful to study the assembly command that will execute that instruction. The assembly code will tell you what registers and values to encode, and all that is left is to then pair the correct opcode and funct values. The table below lists each instruction in assembly. Generally, rd is the destination register, while imm, rs1, and rs2 carry the values needed for each operation. Figure 4.6 and 4.7 list which bits in the instruction correspond to what registers and immediate values. The necessary bit values and positions for an instruction can then be determined by using the assembly command, figures 4.6 and 4.7, and the truth tables for the control unit decoders.

RV32I		RV32A
lw rd, imm(rs1)	lr.w rd, rs1	amoor.w rd, rs2, rs1
sw, rs2, imm(rs1)	sc.w rd, rs2, rs1	amoxor.w rd, rs2, rs1
addi rd, rs1, imm	amoswap.w rd, rs2, rs1	amomax.w rd, rs2, rs1
ori rd, rs1, imm	amoadd.w rd, rs2, rs1	amomin.w rd, rs2, rs1
andi rd, rs1, imm	amoand.w rd, rs2, rs1	

Figure 5.2: Table showing machine language code for each instruction.

RV32I Instructions

Generating proper encoding for RV32I instructions.

There are two forms of the RV32I instructions to make note of: memory accessing—load and store—and arithmetic operation instructions. The memory accessing instructions are in the form of:

“Operation” “destination register” “(offset-value) register-holding-value”

The offset value, determined by the immediate value in the instruction, is a signed integer that provides the ability to choose an address some distance from the source register, rs1.

The load word instruction has a destination, rd, and source register, rs1. The source register has a value in it which gives the address of the desired word. For example, if register t0 has the value 64, the instruction

lw t1, 140(t0)

will load the word stored at address $(60+140=)$ 204 into the register t1.

The store word instruction does not have destination register, but instead two source registers, rs1 and rs2. The combination of rs1 and an immediate value, like in the case of the load instruction, provide the address of the desired location. However, instead of taking the value from that location like load does, store word instead writes the value in rs2 to the register at the address specified by rs1 and imm. Imagine the value 50 was stored in register t1. To write this value to a memory address at location 204, when rs1 already has the value 64 loaded into it, use the store word instruction

sw t1, 140(t0)

to write the value 50 into the register with the byte address of 204. The series of instructions below can combine both the examples of load and store instructions to place the immediate value loaded into a temporary register into the register at byte address 204:

li t1 50 (load the immediate value 50 into the temporary register t1)

li t2 64 (load the immediate value 64 into the temporary register t2)

lw t0 0(t1) (load the word value in register t1 to t0)

sw t0 140(t2) (store the word value in register t0 to the register at address 204)

The non-memory accessing RV32I instructions also use the immediate value to perform their operation, but it is much more obvious how they do so. For example, the instruction

addi rd, rs1, imm

adds the value in register rs1 with the immediate value and stores the result in rd. Creating the desired instructions from this point becomes matching the chosen values for register encodings, the immediate values, the appropriate opcode, and the correct funct3 to the general RV32I instruction format. The registers used by the RISC-V ISA are listed in the table in figure 5.3. The actual register used to create an instruction does not matter in the case of this report, so I chose the three temporary registers x5-x7 (or t0-t2). The 5-bit encoding for those registers can be seen in figure 5.4. [9].

Register	ABI Name	Description	Saver
x0	zero	Zero constant	--
x1	ra	Return address	Callee
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP args/return values	Caller
f12-17	fa2-7	FP args	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Figure 5.3: 32 available registers offered by RISC-V ISA.

5-bit Encoding (rx)	3-bit Compressed Encoding (rx')	Register	ABI Name	Description	Saved by Caller
0	-	x0	zero	hardwired zero	-
1	-	x1	ra	return address	-R
2	-	x2	sp	stack pointer	-E
3	-	x3	gp	global pointer	-
4	-	x4	tp	thread pointer	-
5	-	x5	t0	temporary register 0	-R
6	-	x6	t1	temporary register 1	-R
7	-	x7	t2	temporary register 2	-R

Figure 5.4: 5 bit binary encoding for register numbers.

The finalized version of each instruction is shown in the table below.

Mem Address	Instruction (assembly)	Instruction (binary)	Instruction (hex)	Name	General Instr.
0x0000000	lw t2, 0(t0)	32'b000000000000_00101_000_00111_0000011	32'x00028383	lw	lw rd, imm(rs1)
0x0000004	sw t1, 0(t0)	32'b000000000000_00101_000_00111_0100011	32'x000283A3	sw	sw, rs2, imm(rs1)
0x0000008	addi t2, t0, 1	32'b0000000000001_00101_110_00111_0010011	32'x0012E393	addi	addi rd, rs1, imm
0x000000C	ori t2, t0, 1	32'b0000000000001_00101_111_00111_0010011	32'x0012F393	ori	ori rd, rs1, imm
0x0000010	andi t2, t0, 1	32'b0000000000001_00101_010_00111_0010011	32'x0012A393	andi	andi rd, rs1, imm

Figure 5.5: Encoding for RV32I instructions.

RV32A: lr.w and sc.w Instructions

Generating proper encoding for lr.w and sc.w instructions, as well as additional test cases to validate functionality of atomic instruction signals.

Similar to the RV32I instructions, there are two types of atomic instructions to keep in mind: the load reserve and store conditional, and the arithmetic operation instructions. The method for creating RV32A instructions is the same for RV32I: use the assembly code to understand how each instruction works, then determine the proper encoding of register and immediate values. The values that must still be determined for the RV32A instructions are aq and rl. Lr.w will be the first atomic instruction called, so therefore aq should be high while rl is low to enforce any subsequent atomic instructions to follow the proper protocol.

In order to test the load reserve and store conditional instructions, I will call them twice. Once with a store word instruction coming before the store conditional, and once with lr.w sequentially followed by sc.w. This will test if (1) the proper address is reserved, (2) the flag for the reserved address being accessed is correct, and (3) the store conditional fails when the reserved address flag is tampered with. The fully encoded instructions for load reserve and store conditional are shown in the table below.

Mem Address	Instruction (assembly)	Instruction (binary)	Instruction (hex)	Name	General Instr.
0'x0000014	lr.w t2, t0	32'b00010_1_0_00000_00101_010_00111_0101111	32'x1402A3AF	lr.w	lr.w rd, rs1
0'x0000018	sw t1, 0(t0)	32'b0000000000000_00101_000_00111_0100011	32'x000283A3	sw	sw, rs2, imm(rs1)
0'x000001C	sc.w t2, t1, t0	32'b00011_1_0_00110_00101_010_00111_0101111	32'x1C62A3AF	sc.w	sc.w rd, rs2, rs1
0'x0000020	lr.w t2, t0	32'b00010_1_0_00000_00101_010_00111_0101111	32'x1402A3AF	lr.w	lr.w rd, rs1
0'x0000024	sc.w t2, t1, t0	32'b00011_1_0_00110_00101_010_00111_0101111	32'x1C62A3AF	sc.w	sc.w rd, rs2, rs1

Figure 5.6: Encoding for RV32A lr.w and sc.w instructions.

The remaining RV32A instructions can be encoded in the same format as the preceding instructions. They are shown in the table below.

Mem Address	Instruction (assembly)	Instruction (binary)	Instruction (hex)	Name	General Instr.
0'x0000028	amoswap.w t2, t1, t0	32'b00001_1_0_00110_00101_010_00111_0101111	32'x0C62A3AF	amoswap.w	amoswap.w rd, rs2, rs1
0'x000002C	amoadd.w t2, t1, t0	32'b00000_1_0_00110_00101_010_00111_0101111	32'x0462A3AF	amoadd.w	amoadd.w rd, rs2, rs1
0'x0000030	amoand.w t2, t1, t0	32'b01100_1_0_00110_00101_010_00111_0101111	32'x6462A3AF	amoand.w	amoand.w rd, rs2, rs1
0'x0000034	amoor.w t2, t1, t0	32'b01010_1_0_00110_00101_010_00111_0101111	32'x5462A3AF	amoor.w	amoor.w rd, rs2, rs1
0'x0000038	amoxor.w t2, t1, t0	32'b00100_1_0_00110_00101_010_00111_0101111	32'x2462A3AF	amoxor.w	amoxor.w rd, rs2, rs1
0'x000003C	amomax.w t2, t1, t0	32'b10100_1_0_00110_00101_010_00111_0101111	32'xA462A3AF	amomax.w	amomax.w rd, rs2, rs1
0'x0000040	amomin.w t2, t1, t0	32'b10000_1_0_00110_00101_010_00111_0101111	32'x8462A3AF	amomin.w	amomin.w rd, rs2, rs1

Figure 5.7: Encoding for RV32A operation-based instructions.

5.2.2 Test Bench

The initial testbench implemented the ROM as an array with word addresses, stored in a file called instruction list.dat. To fetch each instruction, the program counter was defined inside the testbench. On each rising clock edge, the program counter was updated to the next byte address and it fetched the next instruction. The instruction was then loaded through a wire to the DUT, which was the instantiation of the top module. This procedure for storing and accessing the ROM was only suitable for initial testing procedures and did not satisfy the PRD. A block of ROM must be included with the control unit to be properly read from. This meant the ROM block and program counter must be moved from the test bench to the design file, leading to a second test bench being created.

The second version of the test bench completely removed the ROM block and the program counter functionality and implemented those in the design file. The top module was again instantiated as the DUT, with its inputs and outputs monitored. A clock was established that alternates every 5 ticks. The program is initialized with reset high to trigger the program counter to zero, then reset is set low for the remainder of the program. The clock is then allowed to run for 170 ticks, completing each instruction.

5.2.3 Results and Discussion

For the initial design, it is enough to check that the log output and waveform followed the expected results from the decoder truth tables based on the instruction. The log output (below) shows that the output signals are indeed correct, however, to take a closer look we need to examine the waveform generated by the design.

Time	PC	Instr	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	ALUControl
0	xxxxxxxx	xxxxxxxx	0	10	1	0	0	0	01	100
5	00000000	00028383	1	00	1	0	1	0	00	000
15	00000004	000283a3	0	01	1	0	0	0	00	000
25	00000008	0012e393	1	00	1	0	0	0	10	000
35	0000000c	0012f393	1	00	1	0	0	0	10	011
45	00000010	0012a393	1	00	1	0	0	0	10	010
55	00000014	1402a3af	1	10	0	0	0	0	00	000
65	00000018	000283a3	0	01	1	0	0	0	00	000
75	0000001c	1c62a3af	1	10	0	0	0	0	00	100
85	00000020	1402a3af	1	10	0	0	0	0	00	000
95	00000024	1c62a3af	1	10	0	0	0	0	00	100
105	00000028	0c62a3af	1	10	0	0	0	0	00	000
115	0000002c	0462a3af	1	10	0	0	0	0	10	000
125	00000030	6462a3af	1	10	0	0	0	0	10	010
135	00000034	5462a3af	1	10	0	0	0	0	10	011
145	00000038	2462a3af	1	10	0	0	0	0	10	111
155	0000003c	a462a3af	1	10	0	0	0	0	10	001

Figure 5.8: Pre-synthesis design file simulation simv log output.

RV32I Instructions

The waveform is broken down by the types of instructions so we can examine each of the important characteristics for each instruction type.

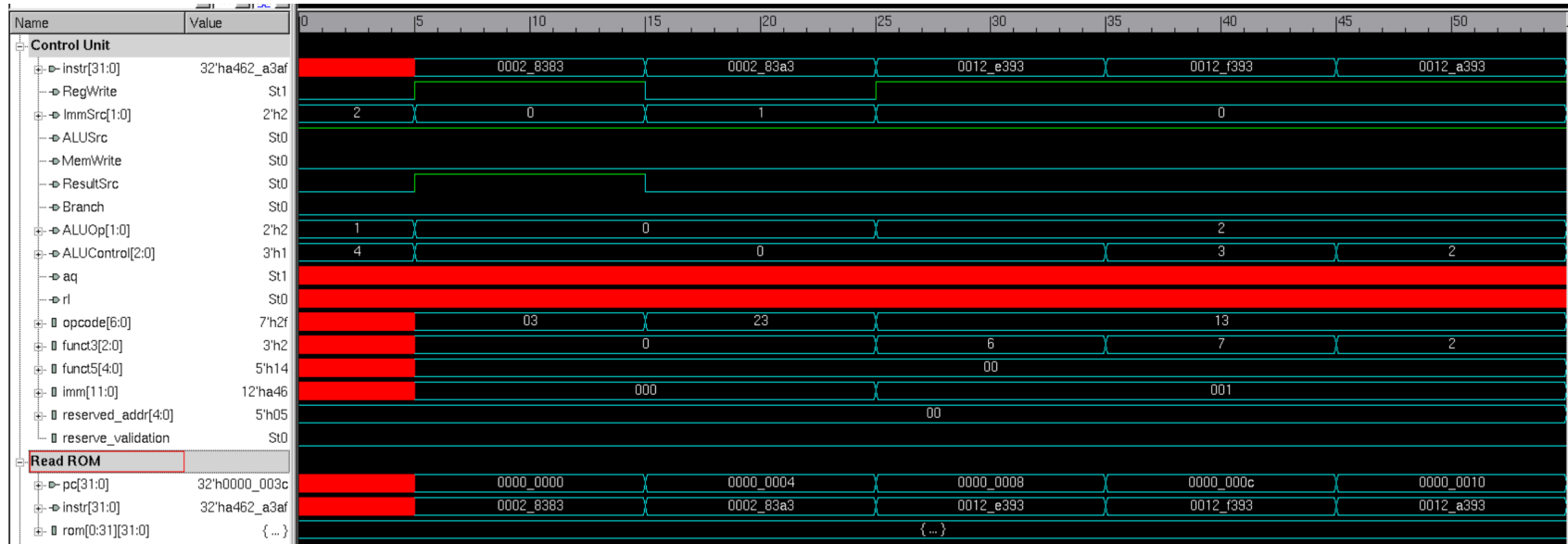


Figure 5.9: Pre-synthesis waveform output for RV32I instructions.

Instructions: lw, sw, addi, ori, andi

The first important observation is that the instruction is being properly read from the test bench into the control unit model, and it is being decoded properly. This is confirmed by the output values of funct3 matching with what is expected from each instruction, based on the Main decoder truth table. It can also be noted that the program counter was initialized with the first rising clock edge, and perpetually incremented by four with each new clock cycle. This matches the desired functionality. Lastly, notice that the outputs for the atomic instructions, aq, rl, reserved address, and reserve validation (flag), remain untouched.

RV32A: lr.w, sc.w Instructions

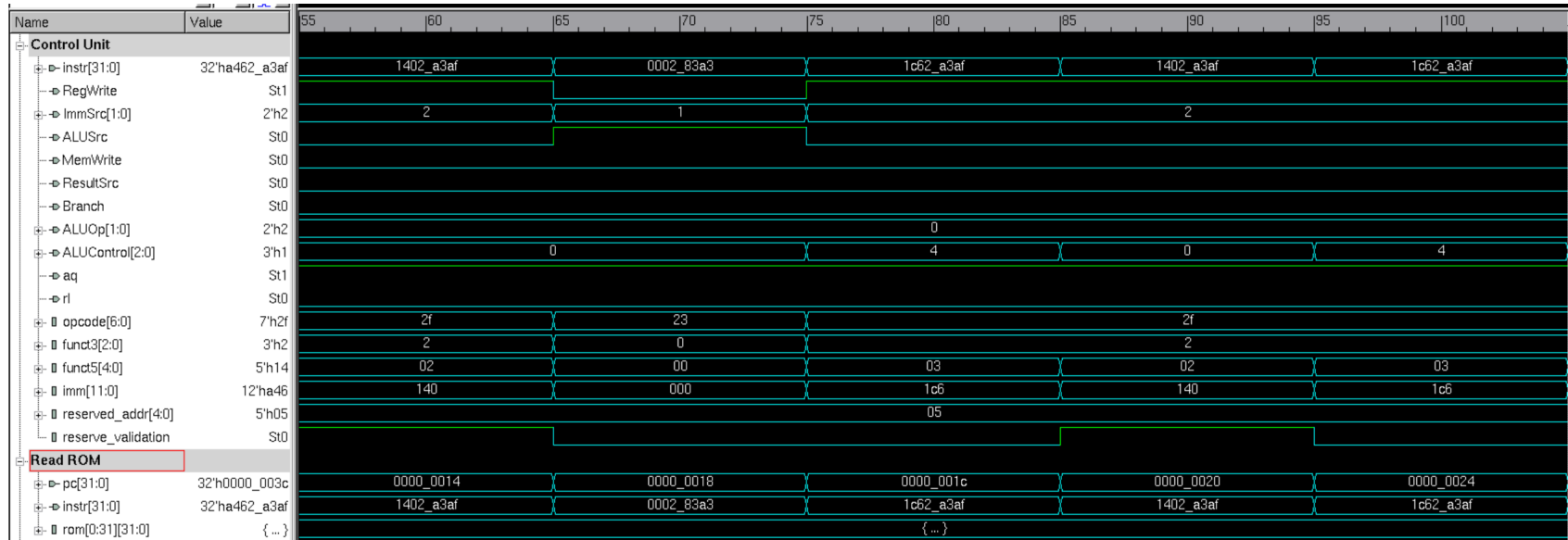


Figure 5.10: Pre-synthesis waveform output for RV32A: `lr.w`, `sc.w` instructions.

Instructions: `lr.w`, `sw`, `sc.w`, `lr.w`, `sc.w`

Once `lr.w` is read, the reserve address is declared as 5, `aq` is set high, and `rl` is set low. This is correct since the instructions set `rd` as x5, or t0, which has a binary encoding of 5'b00101. It is also observed that the reserve validation flag is set high with the first instance of load reserve. This flag is then immediately triggered by the improper access to x5 via the store instruction. The flag is then set high again when load reserve is called once more, and the next instruction, store conditional passes and sets the flag low again since the chain of atomic instructions has been completed.

RV32A: operation-based Instructions

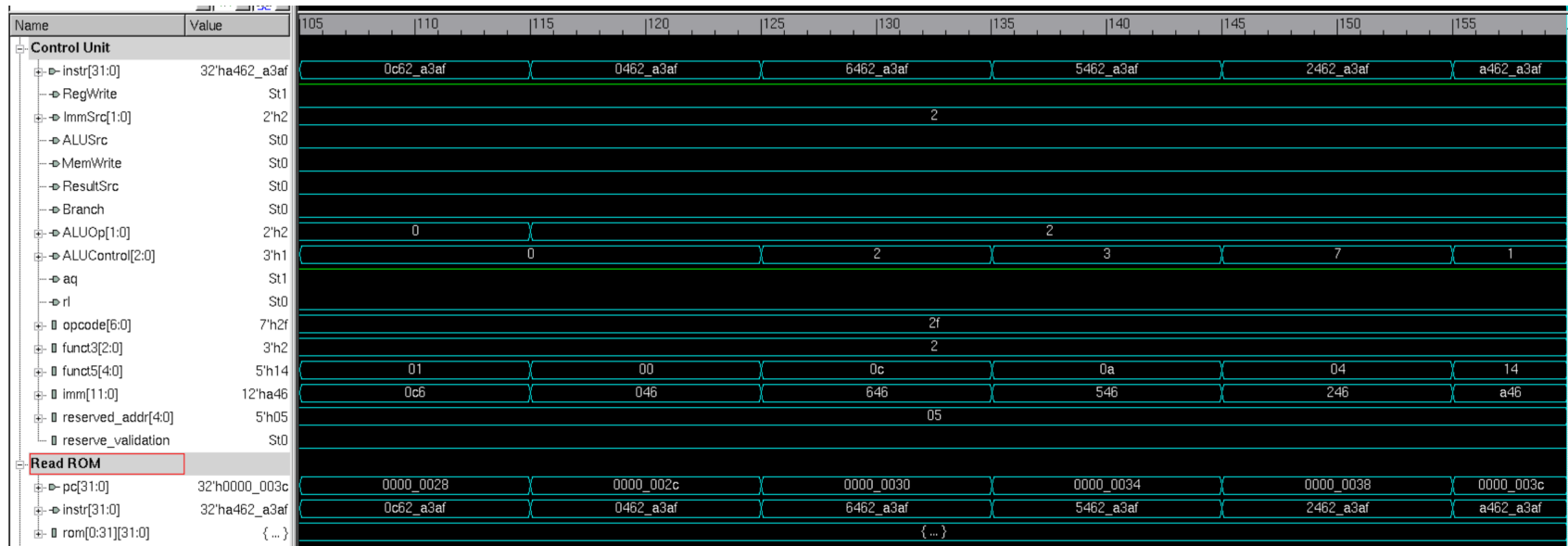


Figure 5.11: Pre-synthesis waveform output for RV32A: operation-based instructions.

Instructions: amoswap.w, amoaddd.w, amoand.w, amoor.w, amoxor.w, amomax.w, amomin.w

The results for the remaining RV32A instructions are again verified by the waveform. The reserved address and validation bit remain unchanged, as expected.

6 Second Design

6.1 Changes

The major changes for the second design involved creating the read rom module, and then creating the interface, harness, and a new testbench file.

The harness file creates the clock port, which is again updating every 5 ticks, a program counter wire to connect the program counter signal from the top module to the interface module. It then instantiates the top module and the testbench program. The testbench program is instantiated with the interface and its TB modport.

The interface file creates two modports. The first is the DUT modport, which contains the signals used by the top module from the design file. The second modport is TB, which contains the signals needed by the testbench. It also defines the clocking block, which for this project only includes the clock established by the harness. A diagram showing the signals in each modport is displayed below.

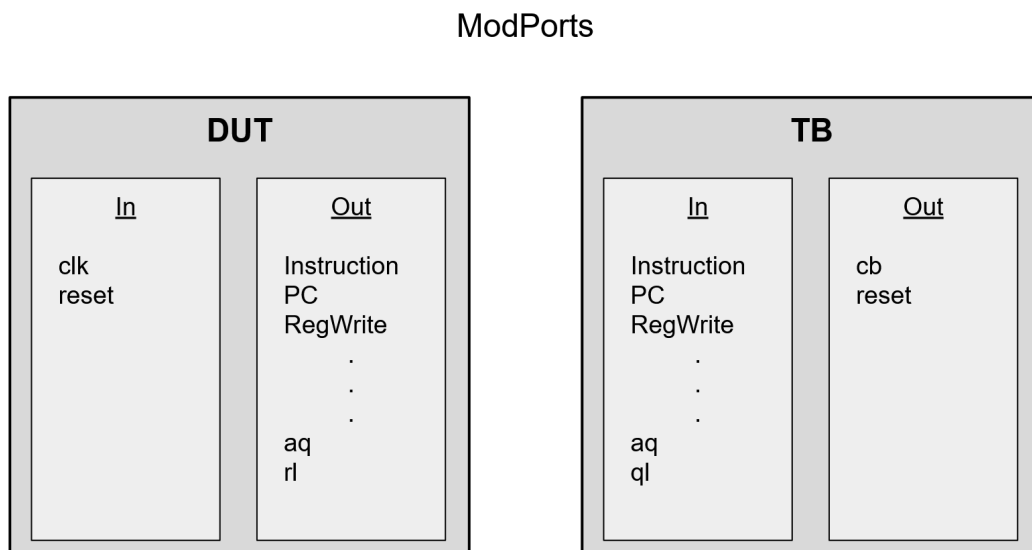


Figure 6.1: Contents of two modports used in System Verilog style testing.

The testbench program is loaded using the TB modport from the interface. It creates a

vdb file for the waveform analysis, and it monitors each of the control signals. It initializes the reset value high to set the program counter to zero. The reset is then set low where it stays, and the program runs for 19 clock cycles. This allows for the program counter to exceed the memory locations of the useful instructions, reaching where default blank instructions are stored. Reset is then set high again to confirm the program counter is functioning properly. A diagram showing the instantiation and connection of the programs, interface, harness, modports, and ports is shown below.

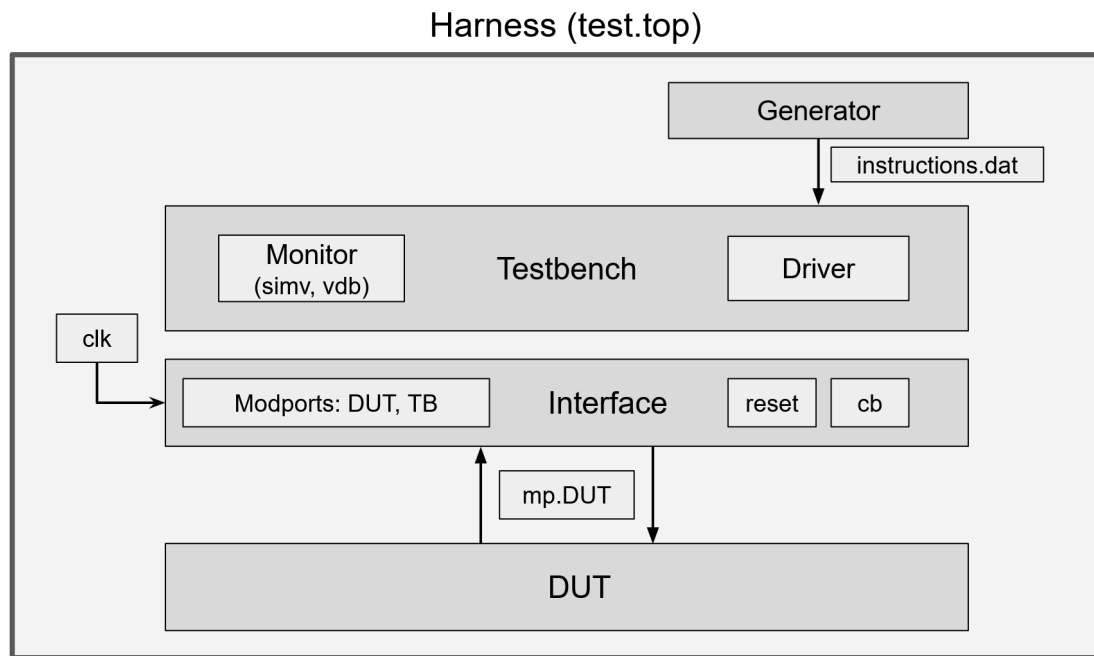


Figure 6.2: Block diagram showing connections between and instantiates of the testing environment. Includes harness, interface, DUT, and testbench.

6.2 Validation

6.2.1 Coverage

In Verilog verification, coverage metrics are used to quantify how well your testbench exercised the design under test. Line, conditional, toggle, and path coverage each provide a different lens into what specifically is being exercised during the simulation of the DUT. Together they are used to enforce confidence in the behavior of RTL logic under a significant number of test cases. Broadly, line coverage measures which lines of source code were executed in simulation. Conditional coverage measures what Flow Control statements, like if statements, are executed, and whether they were executed for true, false, or both. Toggle coverage tracks the transition between states. This is a good measure to see if your test will ensure behavior on both the rising and falling edge of each signal. Lastly, path coverage defines every possible path through the logic embedded in your system and analyzes how much

of each path was traversed. It is not necessary for any of these measures to be 100 percent, depending on the design, but higher test scores that make sense under your conditions are a very good way to build confidence in your design at this early stage. It is important to note, however, that coverage simply that the system reacted to a set of test cases; coverage does not measure how the system reacts to the cases. Therefore, it is good for verifying your design is good enough to continue forward, but it is not a guarantee of proper functionality.

The Unified Reporting Generator (URG) Report combines the four tests into one report. The results for the pre-synthesis coverage tests can be seen in figure 6.3. The top-level results for each test show that line, toggle, and path scores were not 100 percent, while the conditional test result was. These results are not only passable, but actually provide good validation about the testing methods implemented. To further examine each test, we must delve into each module of the design, displayed in figure 6.4.

The interface (control unit if) and harness (contr unit test top) both serve to instantiate and connect control signal blocks and ports by defining modports. It is therefore expected to have such a low toggle score. Meanwhile, the harness's only lines of code that would be covered by the line test are those establishing the clock, therefore it is correct to get a score of 100 for the line test.

The scores of the design file modules are also correct. To see this, we examine them individually and look at the port details and the paths traversed. The low toggle rate in both modules makes sense because their inputs were the instruction, which had many bits that remained unchanged. Firstly, the two least significant bits of the program counter are never changed since they are ignored when determining the address of the ROM block. Second, certain bits of the instructions do not change, like the register addresses, which was done on purpose for simplicity of test cases. This can be seen in the port details diagram in figure 6.5 which shows all the bits that remain unchanged for every instruction.

Total Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	PATH
86.47	96.97	100.00	63.90		85.00

Hierarchical coverage data for top-level instances

SCORE	LINE	COND	TOGGLE	FSM	PATH	NAME
86.47	96.97	100.00	63.90		85.00	control_unit_test_top

Total Module Definition Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	PATH
86.47	96.97	100.00	63.90		85.00

Figure 6.3: Test scores for pre-synthesis coverage report.

NAME	SCORE	LINE	COND	TOGGLE	FSM	PATH
control_unit_if	62.50			62.50		
control_unit_test_top	67.65	100.00		35.29		
rom_read	81.13	100.00		62.26		
top_module	81.25	100.00		62.50		
control	88.59	95.95	100.00	73.42		85.00

Figure 6.4: Test scores by module for pre-synthesis coverage report.

Port Details				
Name	Toggle	Toggle 1->0	Toggle 0->1	Direction
pc[1:0]	No	No	No	INPUT
pc[6:2]	Yes	Yes	Yes	INPUT
pc[15:7]	No	No	No	INPUT
instr[5:0]	Yes	Yes	Yes	OUTPUT
instr[6]	No	No	No	OUTPUT
instr[9:7]	Yes	Yes	Yes	OUTPUT
instr[11:10]	No	No	No	OUTPUT
instr[15:12]	Yes	Yes	Yes	OUTPUT
instr[16]	No	No	No	OUTPUT
instr[17]	Yes	Yes	Yes	OUTPUT
instr[19:18]	No	No	No	OUTPUT
instr[22:20]	Yes	Yes	Yes	OUTPUT
instr[25:23]	No	No	No	OUTPUT
instr[31:26]	Yes	Yes	Yes	OUTPUT

Figure 6.5: Port details for pre-synthesis URG Report Coverage Analysis.

Examining the control module further, it is important to discuss why the line score is not 100, as well as the path score being only 85. The path coverage report, below, shows that the only paths not covered are the default paths from each case statements. This behavior is expected because every instruction corresponds to a correct datapath, and is also the reason the line score is 95.5 for the control module.

Paths:

-1- -2- -3- -4- -5- -6- -7- -8-

Status

default	-	-	-	-	-	-	-	Covered
7'b0100011	-	-	-	-	-	-	1	Covered
7'b0100011	-	-	-	-	-	-	0	Covered
7'b0000011	-	-	-	-	-	-	-	Covered
7'b0010011	-	-	-	-	-	3'b110	-	Covered
7'b0010011	-	-	-	-	-	3'b111	-	Covered
7'b0010011	-	-	-	-	-	3'b010	-	Covered
7'b0010011	-	-	-	-	-	default	-	Not Covered
7'b0101111	1	-	-	-	-	-	-	Covered
7'b0101111	0	0	-	-	5'b0	-	-	Covered
7'b0101111	0	0	-	-	5'b01100	-	-	Covered
7'b0101111	0	0	-	-	5'b01010	-	-	Covered
7'b0101111	0	0	-	-	5'b00100	-	-	Covered
7'b0101111	0	0	-	-	5'b10100	-	-	Covered
7'b0101111	0	0	-	-	5'b10000	-	-	Covered
7'b0101111	0	0	-	-	default	-	-	Not Covered
7'b0101111	0	1	5'b00011	0	-	-	-	Covered
7'b0101111	0	1	5'b00011	1	-	-	-	Covered
7'b0101111	0	1	5'b1	-	-	-	-	Covered
7'b0101111	0	1	default	-	-	-	-	Not Covered

Figure 6.6: Path Coverage for control unit module.

6.3 Synthesis

6.3.1 Design

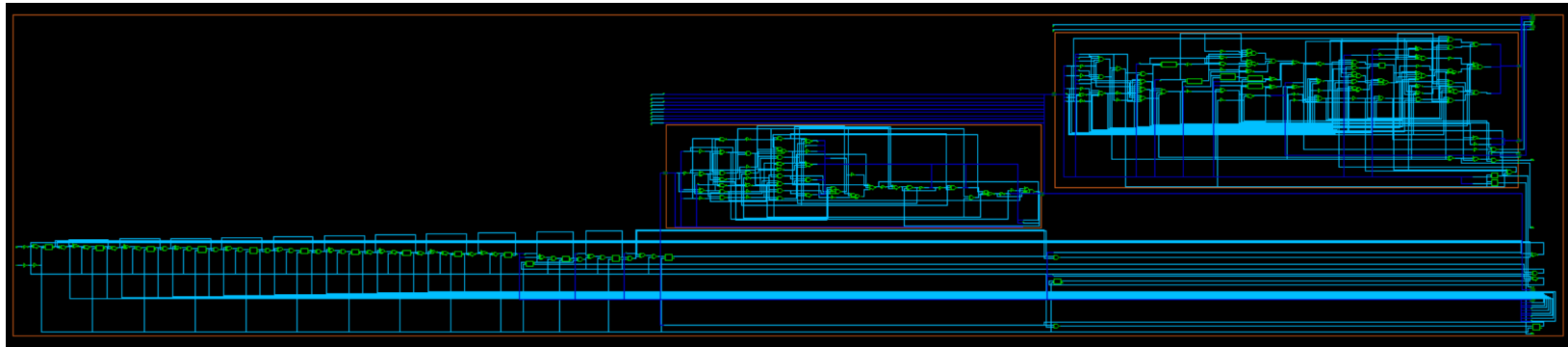


Figure 6.7: Full synthesized module.

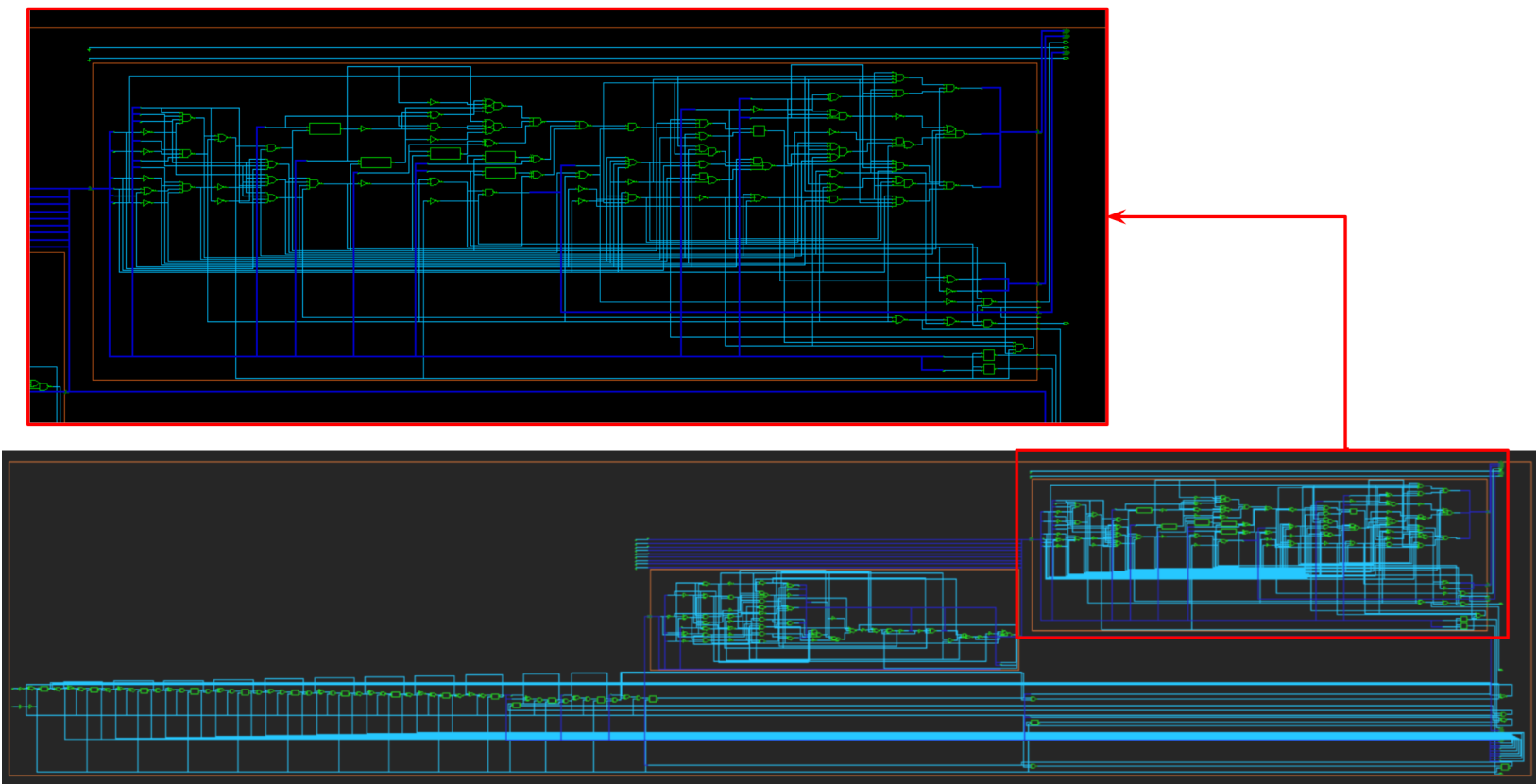


Figure 6.8: Red box highlights a zoomed in view of the control unit module.

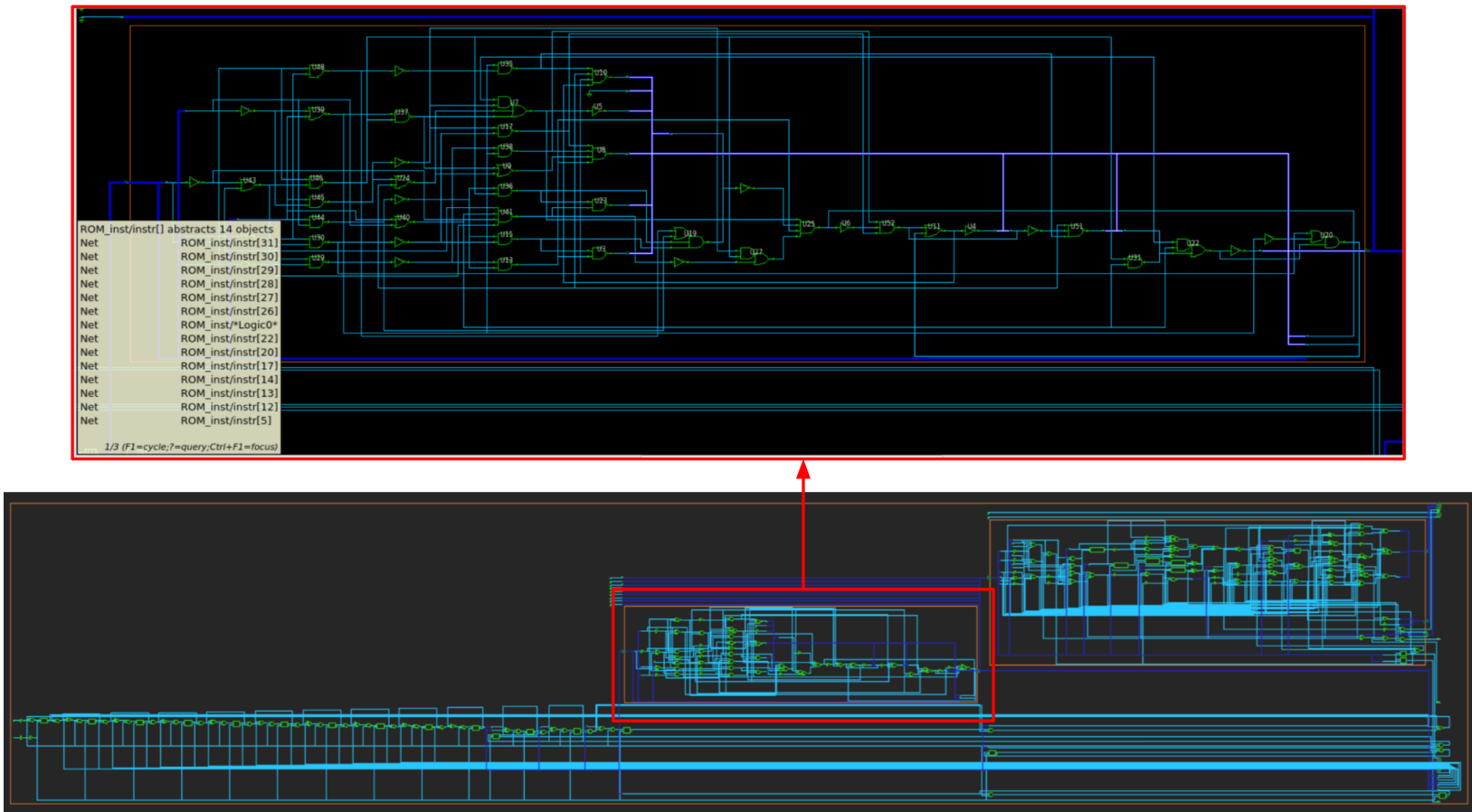


Figure 6.9: Red box highlights a zoomed in view of the ROM read module.

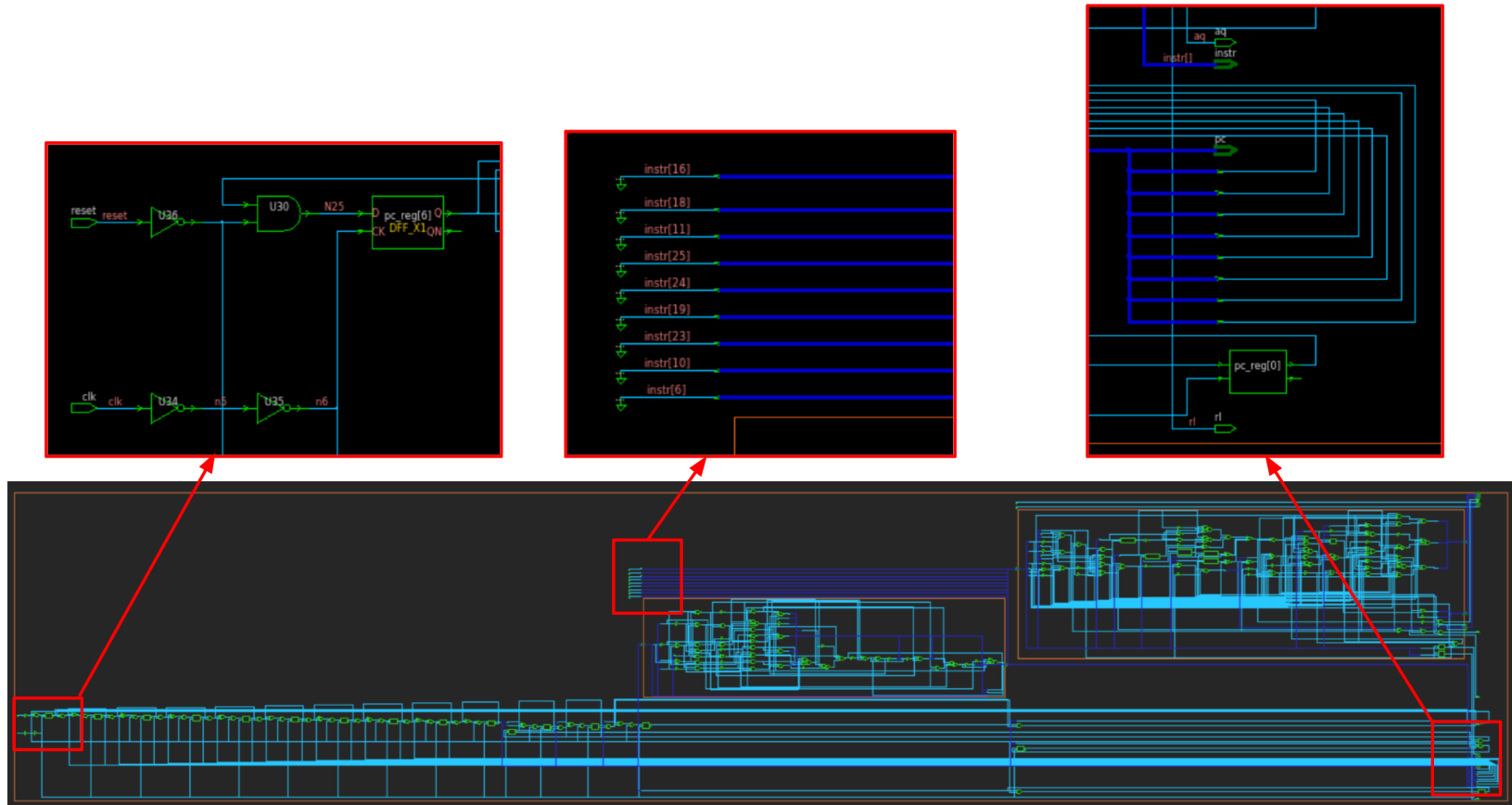


Figure 6.10: Red box highlights a zoomed in view of the ports. The left box shows the clock and reset inputs. The right box shows the instruction, pc, aq, and rl ports. The middle box shows the instruction port being spliced into wires that carry the changing bits needed to encode each instruction.

6.3.2 Log and Waveform

The log and waveform are again identical to the pre-synthesis versions, and are included below. This shows the functionality of the system is the same before and post synthesis, and that all states are properly covered.

Time	reset	PC	instr	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	ALUControl	aq	rl
0	1	xxxx	xXXXxXXx	x	xx	x	0	x	0	xx	xxx	x	x
5	0	0000	00028383	1	00	1	0	1	0	00	000	x	x
15	0	0004	000283a3	0	01	1	0	0	0	00	000	x	x
25	0	0008	0012e393	1	00	1	0	0	0	10	000	x	x
35	0	000c	0012f393	1	00	1	0	0	0	10	011	x	x
45	0	0010	0012a393	1	00	1	0	0	0	10	010	x	x
55	0	0014	1402a3af	1	10	0	0	0	0	00	000	1	0
65	0	0018	000283a3	0	01	1	0	0	0	00	000	0	0
75	0	001c	1c62a3af	1	10	0	0	0	0	00	100	1	0
85	0	0020	1402a3af	1	10	0	0	0	0	00	000	1	0
95	0	0024	1c62a3af	1	10	0	0	0	0	00	100	1	0
105	0	0028	0c62a3af	1	10	0	0	0	0	00	000	1	0
115	0	002c	0462a3af	1	10	0	0	0	0	10	000	1	0
125	0	0030	6462a3af	1	10	0	0	0	0	10	010	1	0
135	0	0034	5462a3af	1	10	0	0	0	0	10	011	1	0
145	0	0038	2462a3af	1	10	0	0	0	0	10	111	1	0
155	0	003c	a462a3af	1	10	0	0	0	0	10	001	1	0
165	0	0040	8462a3af	1	10	0	0	0	0	10	001	1	0
175	0	0044	00000000	0	10	1	0	0	0	01	100	0	0
185	0	0048	00000000	0	10	1	0	0	0	01	100	0	0
195	1	004c	00000000	0	10	1	0	0	0	01	100	0	0
205	0	0000	00028383	1	00	1	0	1	0	00	000	0	0

Figure 6.11: Post-synthesis simv log output.

RV32I Instructions

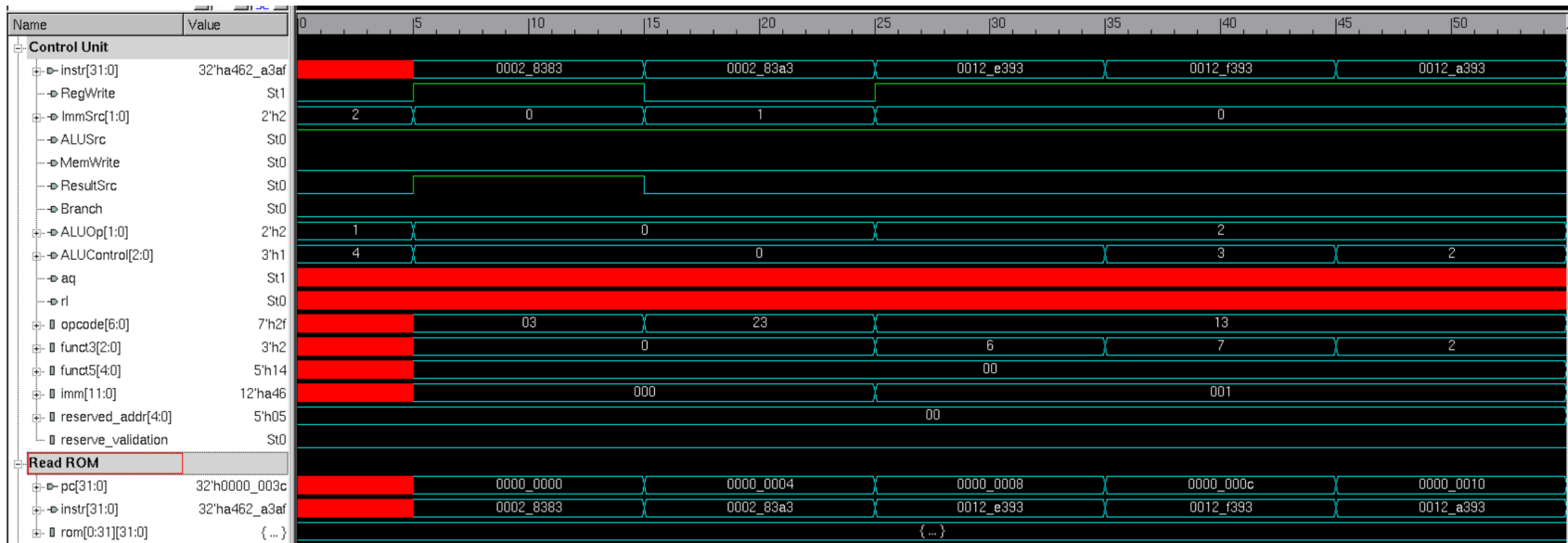


Figure 6.12: Post-synthesis waveform output for RV32I instructions.

Instructions: lw, sw, addi, ori, andi

RV32A: lr.w, sc.w Instructions

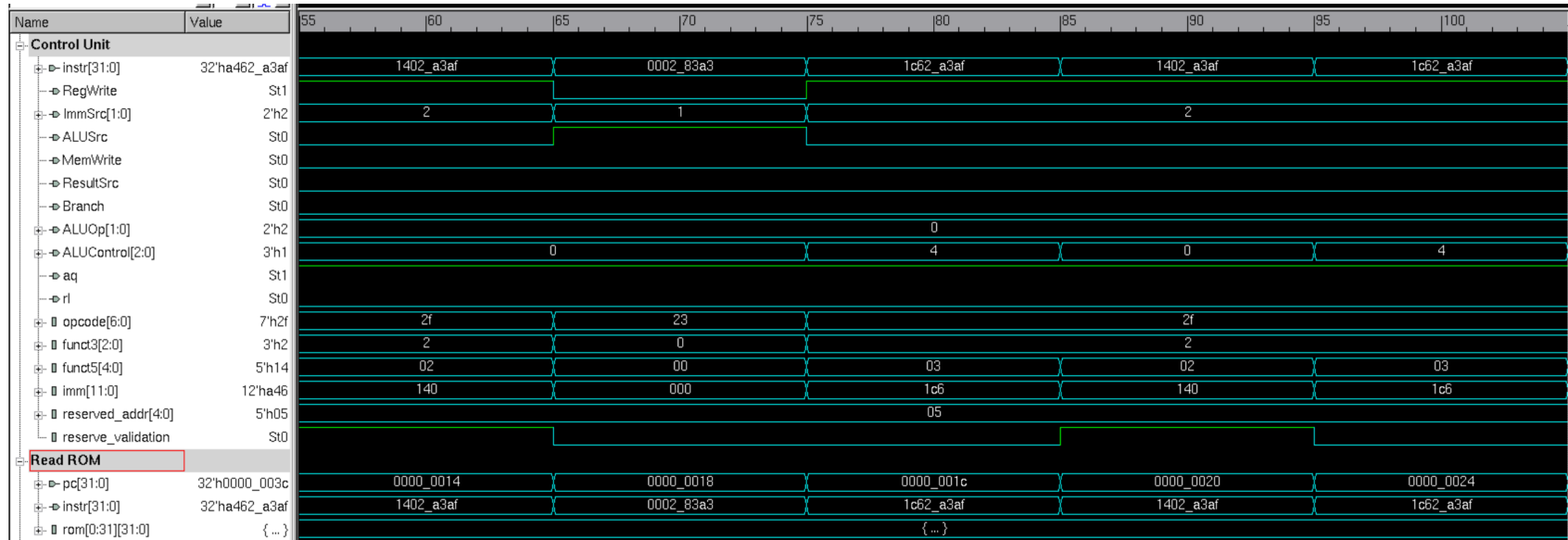
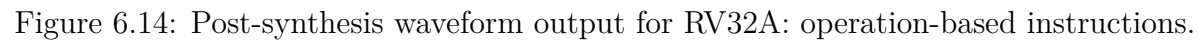


Figure 6.13: Post-synthesis waveform output for RV32A: lr.w, sc.w instructions.

Instructions: lr.w, sw, sc.w, lr.w, sc.w

Leveraging RISC-V Modularity: Design and Implementation of a Control Unit for Single-Cycle Microprocessor

44



Instructions: amoswap.w, amoadd.w, amoand.w, amoor.w, amoxor.w, amomax.w, amomin.w

This last section includes the additional blank default instructions which give a bogus output until reset is set high, where the program counter returns to zero, and the instruction list begins over again.

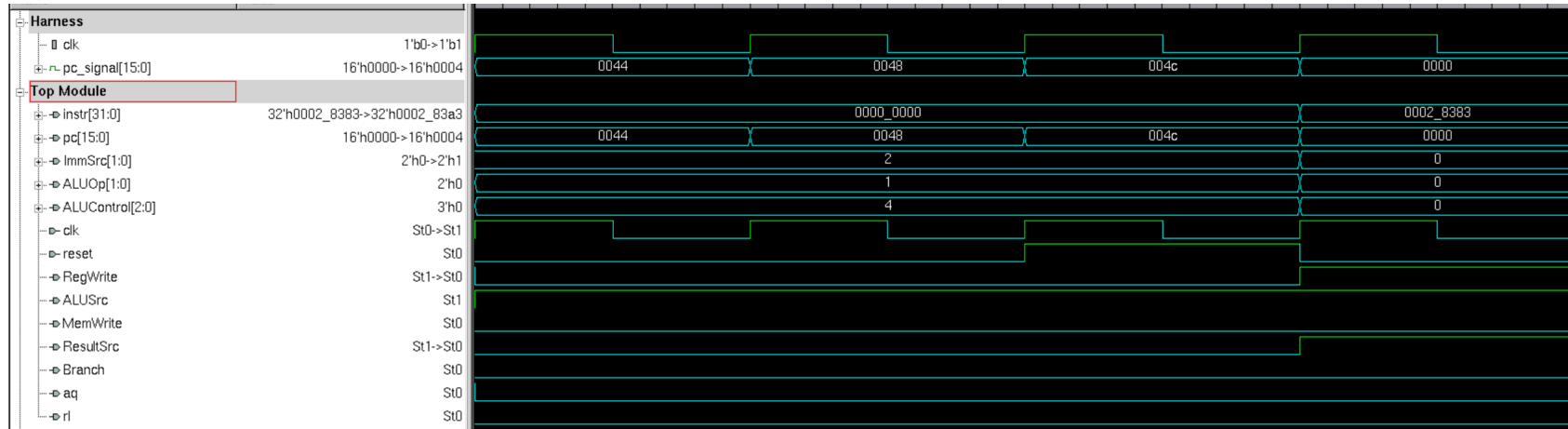


Figure 6.15: Post-synthesis waveform output for additional cases.

7 Conclusion

This project set out to design and implement a specialized control unit for a single-cycle RISC-V microprocessor capable of decoding a subset of RV32I instructions, alongside the complete set of RV32A instructions. The design was approached in two phases—the initial implementation and its subsequent refinement into a semi-final version. In the initial design, the control unit was constructed using two decoders that exploited the modular nature of the RISC-V ISA. By adopting a nested case statement approach, the design accurately produced the required control signals for various instruction types, as verified by log outputs and waveform analysis. During the semi-final design phase, improvements were made to synthesize the system and additional steps were taken to improve the quality of verification. These changes included the addition of a ROM read module, creation of a top module to interconnect the control unit and ROM block, and an adjustment of the default values to aid in synthesis. Testbench development and coverage analysis comprehensively verified that the functionality met the PRD requirements, however, Static Timing Analysis (STA) showed that the capacitance and fanout constraints were violated. The pre-synthesis and post-synthesis comparisons confirm that the critical functionality is retained, even as the design transitioned to a synthesized implementation, although STA strongly indicates that while the design is functionally correct, its physical design does not satisfy the physical constraints.

Bibliography

- [1] S. L. Harris and D. Harris,
Digital Design and Computer Architecture, RISC-V Edition.
Morgan Kaufmann, 2021.
- [2] Wikipedia. “RISC-V.” Accessed: Feb. 21, 2025.
(Jan. 13, 2021),
[Online]. Available: <https://en.wikipedia.org/wiki/RISC-V> (visited on 02/21/2025).
- [3] Wikipedia Contributors. “Instruction set architecture.” Accessed: Feb. 21, 2025.
(Oct. 7, 2019),
[Online]. Available: https://en.wikipedia.org/wiki/Instruction_set_architecture
(visited on 02/21/2025).
- [4] C. Lin. “Instruction format.” Available online; Accessed: Feb. 21, 2025.
(2003),
[Online]. Available: https://en.wikibooks.org/wiki/MIPS_Assembly/Instruction_Formats (visited on 02/21/2025).
- [5] “Risc-v instruction set explanation.” Accessed: Feb. 21, 2025.
(Dec. 1, 2020),
[Online]. Available: <https://fraserinnovations.com/risc-v/risc-v-instruction-set-explanation/> (visited on 02/21/2025).
- [6] “Standard extensions - risc-v - wikichip.” Accessed: Feb. 21, 2025.
(n.d.),
[Online]. Available: https://en.wikichip.org/wiki/risc-v/standard_extensions
(visited on 02/21/2025).
- [7] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic,
“Risc-v instruction set manual, volume i: User-level isa, version 2.0,”
University of California, Berkeley,
Tech. Rep.,
May 2014, Accessed: Feb. 21, 2025.
[Online]. Available: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-54.html>.
- [8] RISC-V Contributors. “Riscv-isa-manual/src/a-st-ext.adoc at main · riscv/riscv-isa-manual.” Accessed: Apr. 14, 2025.
(2017),

[Online]. Available: <https://github.com/riscv/riscv-isa-manual/blob/main/src/a-st-ext.adoc> (visited on 04/14/2025).

- [9] “Registers - risc-v - wikichip.” Accessed: Apr. 14, 2025.

(n.d.),

[Online]. Available: <https://en.wikichip.org/wiki/risc-v/registers> (visited on 04/14/2025).

